

ASPERITAS AI AGENT DEVELOPMENT PLAYBOOK

Vibe Coding + ChatGPT + Codex + GitHub 를 이용한 독자적 에이전트 개발 표준 운영체제

신규 개발자 온보딩 / Codex 실행 프롬프트 / GitHub 검증 루프 / 오픈소스 흡수 / 스킬·워크플로우 확장

Version: 1.0

Date: 2026-06-14

Owner: COO & AI Manager Office

Scope: Asperitas Inc. internal agent-development operating manual

핵심 한 줄

이 문서는 “초안 -> 프롬프트 -> 구현 -> 검증 -> 확인 -> GitHub 커밋/PR -> 학습/개선” 루프를 회사 표준으로 고정하고, 이후 모든 독자적 에이전트 개발을 같은 공장식 프로세스로 복제하기 위한 플레이북이다.

0. Executive Summary - 이 문서의 목적

Asperitas가 앞으로 개발할 모든 독자적 AI 에이전트는 즉흥적인 “AI에게 맡기기”가 아니라, 반복 가능한 엔지니어링 공정으로 운영되어야 한다. 이 플레이북은 현재 진행 중인 asperitas-agent 개발 방식, 즉 초안 작성, ChatGPT를 통한 프롬프트 정교화, Codex 구현, 테스트와 아티팩트 검증, 결과 확인, GitHub 커밋 및 푸시 루프를 표준화한다.

목표는 신규 개발자가 합류했을 때 “무엇을 만들지”, “어떻게 지시할지”, “어떻게 검증할지”, “어떻게 커밋할지”, “다음 에이전트에도 어떻게 재사용할지”를 한 문서만 보고 실행할 수 있게 만드는 것이다.

구분	기존 방식	표준화 후 방식
개발 시작	아이디어를 말하고 바로 구현 요청	Agent Brief -> Scope Lock -> Risk Gate -> MVP Prompt 생성
AI 사용	ChatGPT와 Codex를 상황별로 사용	ChatGPT = 전략/설계/프롬프트/검증, Codex = 코드 구현/테스트/리팩터링
검증	테스트 결과 캡처 후 확인	pytest + artifact verify + eval + git diff + decision log를 필수 체크포인트로 고정
GitHub	GitHub Desktop으로 커밋/푸시	branch/PR/protected main/CI/commit convention까지 포함
오픈소스	좋아 보이는 코드 참고	Scout -> License -> Security -> Benchmark -> Adapt -> Ledger 순서로 흡수
확장	새 채팅에서 성능향상 요청	Skills/AGENTS.md/Workflow Pipeline/Prompt Library를 별도 자산으로 축적

운영 원칙

빠르게 만들되, 검증 없이 믿지 않는다. 많이 가져오되, 라이선스와 보안을 통과하지 못한 오픈소스는 코드베이스에 넣지 않는다. 작은 변경을 자주 커밋하되, main은 항상 복구 가능한 상태로 유지한다.

0.1 CEO-grade 지시문으로 압축하면

- 모든 에이전트 개발은 “제품 요구사항 -> 코드 -> 평가 -> 릴리스”가 아니라 “평가 가능한 요구사항 -> 검증 가능한 코드 -> 재현 가능한 결과 -> 추적 가능한 의사결정”이어야 한다.
- AI는 개발자를 대체하는 것이 아니라 반복 비용을 압축하는 실행 엔진이다. 인간은 방향, 기준, 리스크, 최종 승인을 가져야 한다.
- Codex에게 줄 명령은 구체적이어야 한다. 어떤 파일을 보라, 무엇을 바꾸지 말라, 어떤 테스트를 돌려라, 어떤 형식으로 보고하라가 반드시 포함되어야 한다.
- GitHub는 저장소가 아니라 조직 기억장치다. 코드, 테스트, 의사결정, PR 리뷰, 릴리스 노트가 모두 회사의 운영 지식으로 남아야 한다.
- 오픈소스는 “복붙 재료”가 아니라 “아키텍처 학습과 검증 대상”이다. 라이선스, 보안, 유지보수 상태를 통과한 뒤에만 가져온다.

0.2 공식 문서 기반 전제

이 플레이북은 2026-06-14 기준 공개 공식 문서와 사용자가 현재 구축 중인 asperitas-agent 프로젝트 운영 맥락을 결합해 작성했다. OpenAI는 Codex를 코드 읽기, 편집, 실행을 돕는 coding agent로 설명하며, Codex web/cloud는 별도 클라우드 환경에서 작업을 실행할 수 있다고 안내한다 [S1][S2]. OpenAI의 prompt engineering 문서는 원하는 결과를 안정적으로 얻기 위해 명확한 지시와 반복적 개선이 필요하다고 설명한다 [S3]. Anthropic의 개발자 교육 페이지는 Claude Code, Skills, MCP, RAG, evaluations를 별도 개발 역량으로 다룬다 [S5]. GitHub 문서는 protected branch, dependency review, code scanning, licensing을 협업과 보안의 기본 축으로 설명한다 [S6-S10].

1. 현재 내가 하고 있는 실제 개발 루프

현재 asperitas-agent 개발 과정은 이미 좋은 방향이다. 핵심은 “무작정 코딩”이 아니라 작은 MVP 단위로 요구사항을 잠그고, Codex 가 구현하고, 테스트와 검증 스크립트로 확인하고, GitHub 커밋으로 상태를 고정하는 방식이다. 이 플레이북은 그 루프를 신규 개발자가 따라 할 수 있는 조직 표준으로 만든다.

단계	사용자/ChatGPT/Codex 의 역할	산출물	통과 기준
1. 초안	사용자가 목적, 현재 MVP, 원하는 변경을 자연어로 설명한다. ChatGPT가 이를 개발 가능한 요구사항으로 정리한다.	MVP 초안, scope, non-goals	무엇을 만들지, 무엇을 만들지 않을지 분리됨
2. 프롬프트	ChatGPT가 Codex 용 복불 프롬프트를 만든다. AGENTS.md, skills, constraints, report format 포함.	Codex task prompt	파일 삭제 금지, 작은 변경, 테스트, 보고 형식 명시
3. 구현	Codex가 repo를 읽고 필요한 최소 변경을 수행한다.	코드 변경, 테스트 변경, 문서 변경	기존 behavior 보존, backward compatibility
4. 검증	Codex 또는 로컬에서 pytest, artifact verification, retrieval eval 등 실행.	test log, eval metrics	실패한 테스트가 없고 지표 악화가 설명됨
5. 확인	사용자가 스크린샷/로그를 ChatGPT에 보내고 정상 여부를 판단한다.	검증 판정, next-step decision	커밋 가능/수정 필요/보류 중 하나로 결정
6. GitHub 커밋	GitHub Desktop 또는 CLI로 commit/push.	commit hash, clean working tree	origin/main 과 동기화, 작업트리 clean
7. 다음 MVP	새 작업을 독립된 prompt로 이어간다.	next MVP prompt	이전 결과를 전제로 하지만 변경 범위는 다시 작게 잠금

현재 방식의 강점

이미 MVP 단위, 테스트 중심, 커밋 단위, 대화형 검증을 하고 있다. 이 흐름은 초보자가 AI 코딩을 시작할 때 가장 자주 놓치는 “검증 루프”를 포함한다는 점에서 옳바르다.

현재 방식의 보강점

앞으로는 각 MVP마다 “preflight report”, “acceptance criteria”, “regression risk”, “license/security check”, “decision log”를 더 명시적으로 고정해야 한다. 즉, 개발 속도를 높이면서도 나중에 개발자가 들어와도 재현 가능한 운영 기록을 남겨야 한다.

1.1 현재 asperitas-agent 의 기준 상태

신규 개발자는 현재 프로젝트가 단순 실험이 아니라 단계적 RAG Agent MVP로 축적되고 있다는 점을 이해해야 한다. 지금까지의 핵심 흐름은 Foundation -> Retrieval -> Evaluation Baseline -> Metadata-aware Retrieval -> Structure-aware Chunking -> Failure Analysis / Error Taxonomy 준비로 이어진다.

MVP 단계	의미	신규 개발자가 이해해야 할 것
MVP-001 Foundation	기본 저장소 구조와 관리 문서 기반 마련	repo 구조, 00_ADMIN, 01_RAW_SOURCES, 03_PROCESSED_KB, 09_LOGS 의 역할
MVP-002 Retrieval	기초 검색 구조 구축	검색이 되는 것과 좋은 검색은 다르며, baseline 이 필요함
MVP-002.5 Evaluation Baseline	검색 성능을 수치로 보기 시작	Source match, priority, evidence, section 등 metric 을 보며 변경 영향 판단
MVP-003 Metadata-aware Retrieval	메타데이터 기반 deterministic retrieval	source priority 와 metadata 가 검색 품질을 좌우함
MVP-004 Structure-aware Chunking	heading/section/path 기반 chunk 구조화	문서 구조를 보존해야 RAG 답변 품질이 올라감
MVP-009/010 이후 평가 스택	PASS/FAIL behavior 와 eval stack 안정화	새 기능은 기존 PASS/FAIL 약속을 악화시키면 안 됨
MVP-011 Preflight	Failure Analysis/Error Taxonomy 추가 가능	구현 전 읽기 전용 점검을 통해 변경 위험을

	성 검토	출임
--	------	----

1.2 이 루프를 모든 에이전트에 재사용하는 방식

다른 독자적 에이전트를 만들 때도 이 구조는 바뀌면 안 된다. 바뀌는 것은 도메인과 평가 기준뿐이다. Literature Agent 는 citation accuracy 가 핵심이고, Regulatory Agent 는 source hierarchy 와 최신성 검증이 핵심이며, DBTL Agent 는 실험 단계와 입력/출력 데이터 검증이 핵심이다. 그러나 모든 경우 기본 루프는 같다.

Universal Agent Factory Loop

1. Define Agent Brief

2. Freeze scope and non-goals

3. Create Codex implementation prompt

4. Implement the smallest safe change

5. Add/update tests

6. Run verification commands

7. Review diff and logs

8. Commit and push

9. Record decision

10. Create next MVP prompt

2. 역할 분리: ChatGPT, Codex, GitHub, 개발자

AI 코딩이 망하는 가장 흔한 이유는 한 도구에게 모든 역할을 섞어서 맡기기 때문이다. 이 플레이북에서는 역할을 분리한다. ChatGPT는 설계자, 프롬프트 엔지니어, 검증 컨설턴트다. Codex는 코드베이스 안에서 움직이는 구현 에이전트다. GitHub는 변경 이력과 협업 게이트다. 인간 개발자는 최종 책임자다.

주체	핵심 역할	해야 하는 일	하면 안 되는 일
COO/Product Owner	방향과 기준 설정	비즈니스 목표, moat, scalability, compliance 요구 정의	AI가 만든 결과를 검증 없이 승인
ChatGPT	전략/설계/프롬프트/리뷰	요구사항 분해, Codex prompt 생성, 로그 해석, next step 판단	repo 상태를 보지 않고 변경 완료 단정
Codex	코드 구현/테스트 실행	repo inspect, 최소 변경, pytest, verification, report	광범위 리팩터링, 파일 삭제, 임의 dependency 추가
GitHub	기록/협업/릴리스 게이트	branch, PR, review, action, protected main, issue tracking	검증 안 된 코드를 main에 직접 누적
신규 개발자	실행과 검토	프롬프트 실행, 로그 확인, 실패 수정, 문서화	AI 결과를 “대충 맞겠지”로 처리

2.1 왜 ChatGPT와 Codex를 분리해서 써야 하는가

- ChatGPT는 큰 그림을 잘 잡는다. 즉, MVP 범위, 리스크, acceptance criteria, prompt 구조화, 로그 해석에 강하다.
- Codex는 repo 내부 작업에 강하다. 즉, 파일을 읽고, 변경하고, 테스트하고, diff를 보고, 구현 결과를 정리하는 데 적합하다.
- GitHub는 기억과 검증의 장소다. 즉, commit, PR, branch protection, GitHub Actions가 개발 프로세스를 추적 가능하게 만든다.
- 따라서 “ChatGPT에게 전략을 만들게 하고, Codex에게 구현하게 하고, GitHub로 검증 가능한 이력을 남기는” 3분할이 가장 안전하다.

2.2 표준 의사결정권

결정	권한자	AI의 역할	필수 증거
새 MVP 시작	COO/Product Owner	scope와 prompt 초안 제안	objective, acceptance criteria, target files
파일 구조 변경	Tech Lead 또는 명시 승인	영향 분석과 migration 제안	tree diff, backward compatibility 설명
dependency 추가	Tech Lead + Security Gate	대안 비교와 라이선스/보안 체크	license, vulnerabilities, maintainer health
main merge	Reviewer/Owner	PR summary와 test result 분석	passing CI, reviewed diff, clean report
release	Owner	release notes 생성	version tag, changelog, rollback plan

3. 전체 개발 프로세스 - Step by Step

이 장은 신규 개발자가 실제로 따라 할 수 있는 실행 순서다. 각 단계는 산출물과 통과 기준이 있어야 하며, 통과 기준이 없으면 다음 단계로 넘어가지 않는다.

Step 0. 문제를 Agent Brief 로 고정

- 무엇을 자동화할지 한 문장으로 정의한다.
- 누가 사용할지, 어떤 입력을 받을지, 어떤 출력을 내야 하는지 쓴다.
- 성공 기준을 수치나 검증 가능한 조건으로 바꾼다.
- 이번 MVP 에서 하지 않을 것을 명시한다.

Agent Brief Template

```
Agent Name:
Business Objective:
Primary User:
Input Sources:
Expected Output:
MVP Scope:
Non-goals:
Risk Constraints:
Success Metrics:
Required Tests:
Decision Owner:
```

Step 1. Repository 와 GitHub 연결 확인

- GitHub repo 가 private 인지 public 인지 확인한다. 회사 내부 코드는 기본적으로 private 가 원칙이다.
- 로컬 working tree 가 clean 인지 확인한다.
- origin/main 과 동기화 상태를 확인한다.
- GitHub Desktop 을 쓰면 Fetch origin, Pull, No local changes 상태를 먼저 본다.
- CLI 를 쓰면 git status, git branch, git log -1, git rev-parse HEAD 를 확인한다.

```
git status
git branch --show-current
git fetch origin
git rev-parse HEAD
git rev-parse origin/main
git status --short
```

Step 2. AGENTS.md 와 Skill 지시문을 먼저 읽게 하기

- Codex 에게 “Use AGENTS.md and relevant .agents/skills instructions”를 항상 첫 줄에 넣는다.
- 프로젝트 표준, 금지사항, 테스트 명령, 보고 형식을 AI 가 매번 기억하게 만든다.
- 새 에이전트 프로젝트를 만들 때도 AGENTS.md 는 repo 최상단에 둔다.
- 반복 작업은 .agents/skills/<skill-name>/SKILL.md 로 분리한다.

Use AGENTS.md and the relevant .agents/skills instructions.

Task:
[구현할 작업]

Context:
- Current repo: [repo name]
- Current MVP stage: [MVP-xxx]
- Target files: [known files or inspect first]

Constraints:
- Do not delete files.
- Make the smallest safe change.
- Preserve backward compatibility.
- Add or update tests.
- Run pytest.
- Run artifact verification.
- Run eval if retrieval/scoring/generation is affected.
- Update docs if behavior changes.

Report:
1. objective
2. files changed
3. tests run
4. eval metrics before/after
5. risks

Step 3. Read-only preflight 먼저 돌리기

- 큰 변경 전에는 구현하지 말고 repo 상태와 영향 범위를 읽기 전용으로 분석시킨다.
- 어떤 파일을 바꿔야 하는지, 어떤 테스트가 깨질 가능성이 있는지, 어떤 metric 이 악화될 수 있는지 먼저 보고받는다.
- preflight 결과가 명확하지 않으면 구현 prompt 를 바로 주지 않는다.

Use AGENTS.md and relevant skills.

Task:

Run a read-only preflight for [feature]. Do not edit, create, delete, commit, push, merge, rebase, reset, or ingest files.

Inspect:

- Current branch and HEAD
- Relevant modules
- Existing tests
- Existing eval scripts
- Data/artifact dependencies

Report:

1. current repo state
2. affected files
3. proposed minimal implementation plan
4. tests/evals to run
5. regression risks
6. explicit go/no-go recommendation

Step 4. 구현 프롬프트 작성

- preflight 에서 나온 파일만 target 으로 잡는다.
- 변경 범위를 MVP 수준으로 작게 유지한다.
- 테스트 추가/수정, 문서 업데이트, 검증 명령을 prompt 에 넣는다.
- 보고 형식을 고정해 결과를 비교 가능하게 만든다.

Task:

Implement [MVP/feature name] based on the read-only preflight.

Implementation requirements:

- Keep existing PASS/FAIL behavior unchanged unless explicitly required.
- Additive change only where possible.
- Preserve schema backward compatibility.
- Add tests for new behavior and regression tests for old behavior.
- Avoid broad refactors.
- Do not introduce new dependencies unless necessary; justify any dependency.

Verification:

- Run pytest.
- Run scripts/verify_artifacts.py if artifacts are affected.
- Run retrieval eval if retrieval/chunking/scoring/generation is affected.

Report:

1. objective
2. implementation summary
3. files changed
4. tests run with exact result
5. eval metrics before/after
6. known risks
7. suggested commit message

Step 5. Codex 구현 결과를 검증

- Codex 가 성공이라고 말해도 로그를 본다.
- pytest passed, artifact ok, eval metric 변화를 확인한다.
- git diff 가 의도한 범위 밖으로 나갔는지 확인한다.
- 변경 파일이 너무 많으면 “왜 필요한지” 설명을 요구한다.

```
git status --short
git diff --stat
git diff --name-only
pytest
python scripts/verify_artifacts.py
python scripts/run_retrieval_eval.py
```

Step 6. ChatGPT 에게 결과 화면/로그를 보내 판정

- 테스트 결과 화면, changed files, eval metric, GitHub Desktop 상태를 보낸다.
- ChatGPT 에게 “커밋 가능/수정 필요/보류” 중 하나로 판정하게 한다.

- 실패가 있으면 Codex 용 fix prompt 를 다시 생성한다.
- 성공이면 commit message 와 next MVP prompt 를 생성한다.

Result Review Prompt

Here are the Codex results for [MVP].
[Paste logs/screenshots/changed file list]

Decide:

1. Is this safe to commit?
2. What exact risks remain?
3. What commit message should I use?
4. What is the next MVP prompt?

Step 7. GitHub commit/push

- GitHub Desktop 기준: Changes 탭에서 파일 확인 -> summary 작성 -> Commit to main 또는 feature branch -> Push origin.
- 조직 개발에서는 main 직접 커밋보다 feature branch + PR 이 원칙이다.
- 혼자 빠르게 MVP 를 쌓는 초기에는 main direct commit 도 가능하지만, clean state 와 test log 가 있어야 한다.
- commit message 는 MVP 번호와 기능을 포함한다.

```
git add [files]
git commit -m "MVP-xxx: implement [feature]"
git push origin [branch]
```

```
git status
git log -1 --oneline
```

Step 8. Decision log 업데이트

- 왜 이 변경을 했는지, 어떤 metric 이 나왔는지, 어떤 리스크가 남았는지 기록한다.
- 나중에 개발자가 들어오면 commit 만 보고는 맥락을 모른다. decision log 가 맥락을 제공한다.
- 특히 RAG, 규제, 생물학 데이터, 라이선스 관련 결정은 반드시 근거를 남긴다.

Decision Log Entry

Date:
MVP:
Objective:
Files Changed:
Tests/Evals:
Metrics Before:
Metrics After:
Decision:
Risks:
Next Step:
Owner:

4. GitHub 운영 표준

GitHub 는 코드 저장소를 넘어 개발 프로세스의 법적/기술적/조직적 증거 시스템이다. 신규 개발자는 GitHub 를 “올리는 곳”이 아니라 “검증된 변경만 살아남는 운영 게이트”로 이해해야 한다.

4.1 저장소 공개 범위

상황	권장 설정	이유
회사 내부 에이전트, API 키, 데이터 파이프라인 포함	Private	지식재산, 비공개 데이터, credential 노출 방지
논문 재현 코드, 공개 데모, 채용 포트폴리오	Public 가능	라이선스, secret scan, 문서화 완료 후 공개
외부 협업자 1-3 명만 필요	Private + collaborator 초대	필요한 사람에게만 read/write 권한 부여
오픈소스 프로젝트로 전환	Public + LICENSE + SECURITY.md + CONTRIBUTING.md	재사용 권리와 보안 신고 절차 명시

4.2 Branch 전략

초기 1 인 개발은 main direct commit 이 빠르다. 그러나 신규 개발자가 들어오면 main 직접 푸시는 줄이고 feature branch + PR + required checks 로 전환해야 한다. GitHub 의 protected branch 기능은 중요한 브랜치 삭제, force push, failing status check 병합을 제한할 수 있다 [S6].

브랜치	용도	규칙
main	항상 복구 가능한 기준선	테스트 통과한 변경만 들어감. 가능하면 protected branch 적용.
feature/mvp-xxx-name	새 MVP 구현	작고 독립적인 변경. PR 로 병합.
fix/test-failure-name	실패 테스트 수정	원인 분석, regression test 포함.
experiment/name	실험적 비교	main 병합 전 반드시 정리, 불필요하면 폐기.
release/vx.y	배포 준비	tag, changelog, rollback plan 포함.

4.3 Commit Message 규칙

Commit message format

MVP-004: add structure-aware chunk metadata
MVP-011: add read-only failure taxonomy preflight
fix: preserve retrieval PASS/FAIL behavior in eval runner
test: add regression coverage for chunk section audit
docs: update developer onboarding workflow
security: add dependency review notes for OSS intake

4.4 Pull Request Template

```
# PR Summary

## Objective
[What this PR changes and why]

## Scope
- [ ] Code
- [ ] Tests
- [ ] Docs
- [ ] Data/artifacts
- [ ] Eval scripts

## Files Changed
[List important files]

## Verification
- [ ] pytest passed
- [ ] artifact verification passed
- [ ] retrieval eval run if applicable
- [ ] no unexpected file deletion
- [ ] no new dependency without review

## Metrics
Before:
After:
Explanation:
```

```
## Risks
[Regression/security/data/license risks]

## Rollback Plan
[How to revert safely]
```

4.5 GitHub Actions 최소 CI

협업이 시작되면 로컬에서만 pytest를 돌리는 방식은 부족하다. GitHub Actions로 최소한 lint/test/security check를 자동화해야 한다. 단, CI 추가도 작은 MVP로 진행한다.

```
name: ci
on:
  pull_request:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: python -m pip install -U pip
      - run: pip install -e .[dev]
      - run: pytest
      - run: python scripts/verify_artifacts.py
```

4.6 보안 기본값

- GitHub secret scanning 또는 로컬 secret scan을 켜다.
- 환경변수, API 키, 토큰은 절대 repo에 커밋하지 않는다.
- 새 dependency는 dependency review 또는 유사 도구로 확인한다. GitHub dependency review는 PR에서 dependency 변화와 보안 영향을 이해하는 데 도움을 준다 [S8].
- Code scanning은 취약점과 코드 오류를 찾기 위한 기능이다 [S9]. 초기에는 CodeQL default setup부터 고려한다.
- 오픈소스 코드를 가져올 때 license 파일이 없으면 원칙적으로 직접 복사하지 않는다. GitHub는 공개 repo가 진짜 오픈소스가 되려면 사용, 변경, 배포 권리를 주는 license가 필요하다고 설명한다 [S7].

5. 오픈소스 흡수 프로토콜 - “좋은 거 다 긁어오기”의 안전한 방식

오픈소스를 적극적으로 조사하는 것은 맞다. 그러나 “좋은 코드니까 복사”는 위험하다. 회사 코드베이스에 들어오는 모든 외부 코드는 지식 재산, 라이선스, 보안, 유지보수 리스크를 만든다. 따라서 좋은 오픈소스는 먼저 배우고, 필요한 패턴만 검증 후 흡수한다.

핵심 원칙

Open-source intake 는 “Scrape”가 아니라 “Scout -> Score -> Sandbox -> Verify -> Adapt -> Document”다. 코드를 무조건 가져오는 것이 아니라, 아키텍처 패턴과 테스트 전략을 학습하고 필요한 부분만 라이선스에 맞게 통제해서 적용한다.

5.1 오픈소스 탐색 기준

평가 항목	질문	통과 기준
License	MIT/Apache-2.0/BSD 등 상업적 사용 가능한가? Copyleft 의무는 있는가?	LICENSE 명확, 회사 정책과 충돌 없음
Maintainer Health	최근 commit, release, issue response 가 있는가?	최근 활동, 열린 취약점 방치 없음
Security	known CVE, malicious package, secret leakage 이슈가 있는가?	dependency scan 통과
Tests	테스트가 존재하고 CI 가 도는가?	핵심 기능 테스트 존재
Architecture Fit	우리 repo 구조와 맞는가? 과도한 dependency 가 필요한가?	작은 단위로 적용 가능
Data/Model License	모델 weight, dataset, embeddings 사용 조건이 맞는가?	상업적 사용/재배포 조건 확인
Community Signal	stars 보다 issue quality 와 docs 가 좋은가?	문서와 examples 가 재현 가능
Replacement Cost	직접 구현하는 것이 더 안전한가?	복잡도가 높을 때만 도입

5.2 오픈소스 흡수 단계

1. Scout: GitHub, papers, docs, benchmarks 에서 후보 repo 를 찾는다.
2. Shortlist: 목적별 3-5 개 후보로 줄인다. 예: RAG indexing, eval framework, chunking, citation parser, agent orchestration.
3. License Gate: LICENSE, NOTICE, dependency license 를 확인한다.
4. Security Gate: dependency review, code scanning, secret scan, suspicious package check 를 수행한다.
5. Sandbox: 별도 실험 브랜치 또는 별도 repo 에서 돌려본다. main 에 바로 넣지 않는다.
6. Benchmark: 우리 테스트셋 또는 synthetic fixture 로 성능/속도/품질을 확인한다.
7. Adapt: 코드를 그대로 복사하기보다 필요한 구조, 알고리즘, 테스트 패턴만 재구현한다.
8. Ledger: 어떤 repo 에서 무엇을 참고했는지 출처와 license 를 기록한다.
9. Review: PR 에서 license/security/architecture reviewer 가 승인한다.
10. Commit: 통과한 변경만 회사 repo 에 반영한다.

5.3 OSS Intake Ledger 템플릿

OSS Intake Ledger

```

Candidate name:
Repository URL:
Commit/Release reviewed:
Primary purpose:
License:
Commercial use allowed? yes/no/unclear
Dependencies introduced:
Security scan result:
Test/benchmark result:
Code copied? yes/no
If copied, exact files/functions:
If not copied, pattern learned:
Attribution/NOTICE requirement:
Decision: reject / learn only / sandbox / adopt
Reviewer:
Date:
  
```

5.4 “복사”보다 “패턴 흡수”가 우선인 이유

- 우리 회사의 moat 는 남의 repo 를 많이 붙이는 것이 아니라, Asperitas 데이터, eval, 운영 지식, 규제/바이오 도메인 구조를 자체 시스템에 녹이는 것이다.

- 외부 코드는 빠른 시작에는 좋지만 장기 유지보수와 IP 리스크를 키운다.
- 좋은 오픈소스는 테스트 방식, 모듈 경계, CLI 설계, config 관리, docs 구조를 배우는 데 가장 유용하다.
- 라이선스가 불명확하거나 보안 상태가 나쁘면 코드가 좋아도 폐기한다.

6. Codex 프롬프트 운영법

Codex 프롬프트의 품질이 개발 품질을 결정한다. 좋은 프롬프트는 길기만 한 것이 아니라, 목표, 현재 상태, 제약, 검증, 보고 형식이 명확하다. OpenAI의 prompt engineering 문서도 원하는 결과를 안정적으로 얻기 위해 명확한 지시와 반복 개선이 중요하다고 설명한다 [S3].

6.1 Codex 프롬프트 기본 구조

블록	목적	예시
Instruction Header	AGENTS.md와 skill 지시문을 읽게 함	Use AGENTS.md and relevant .agents/skills instructions.
Task	구체적 구현 목표	Implement MVP-012 confidence scoring for retrieval evidence.
Context	repo, branch, MVP, target files	Current repo: asperitas-agent; target files: inspect first.
Constraints	금지사항과 안전장치	Do not delete files. Preserve backward compatibility.
Verification	테스트와 eval 명령	Run pytest, verify_artifacts, retrieval eval.
Report	결과 보고 형식	objective, files changed, tests run, metrics, risks.

6.2 구현용 Master Prompt

```
Use AGENTS.md and the relevant .agents/skills instructions.

Task:
[Write the exact MVP or feature to implement]

Context:
- Current repo: asperitas-agent
- Current MVP stage: MVP-[number]
- Target files: inspect first unless listed below
- Previous baseline: [paste metrics or status]

Constraints:
- Do not delete files.
- Make the smallest safe change.
- Preserve backward compatibility.
- Do not weaken existing PASS/FAIL behavior.
- Add or update tests.
- Avoid new dependencies unless clearly justified.
- Update docs only if behavior changes.

Verification:
- Run pytest.
- Run artifact verification.
- Run retrieval eval if retrieval/chunking/scoring/generation is affected.
- Show exact command outputs or concise summaries.

Report:
1. objective
2. files changed
3. implementation summary
4. tests run
5. eval metrics before/after
6. risks and rollback plan
7. suggested commit message
```

6.3 실패 테스트 수정 Prompt

```
Use AGENTS.md and relevant skills.

Task:
Fix the failing tests from the previous implementation without expanding scope.

Failure context:
[Paste failing test output]

Constraints:
- Do not rewrite unrelated modules.
- Do not delete tests to make the suite pass.
- Prefer fixing the production code if the test captures intended behavior.
- If the test is wrong, explain why and update it minimally.
- Preserve existing behavior outside the failure.
```

Verification:

- Re-run the failing test first.
- Then run full pytest.
- Run artifact/eval verification if affected.

Report:

1. root cause
2. files changed
3. test result before/after
4. residual risk

6.4 Read-only Architecture Review Prompt

Use AGENTS.md and relevant skills.

Task:
Perform a read-only architecture review for [agent/module]. Do not edit files.

Analyze:

- Current module boundaries
- Data flow
- Test coverage
- Failure modes
- Security/license/data risks
- Opportunities for simplification
- Next 3 MVP candidates

Report:

1. architecture map
2. bottlenecks
3. risky assumptions
4. recommended next MVP
5. why not to do the tempting but unsafe alternatives

6.5 새 채팅 이어받기 Prompt

토큰이 길어지거나 새 주제로 넘어갈 때는 새 채팅을 열고 아래 prompt를 사용한다. 이것은 “기억 압축”이다. 단, 실제 repo 상태는 Codex/GitHub에서 다시 확인해야 한다.

We are continuing the Asperitas AI agent development workflow.

Project:

- Repo: asperitas-agent
- Branch: main unless otherwise stated
- Current MVP stage: [MVP-xxx]
- Latest known commit: [hash if known]
- Latest known status: [clean / modified / tests passed / pending]

Operating loop:
초안 -> ChatGPT prompt -> Codex implementation -> pytest/artifact/eval verification -> user review -> GitHub commit/push -> next MVP.

Rules:

- Use AGENTS.md and .agents/skills instructions.
- Smallest safe change.
- No file deletion unless explicitly approved.
- Preserve backward compatibility.
- Add/update tests.
- Report objective, changed files, tests, metrics, risks.

Current task:
[Paste the next task]

Ask me for screenshots/logs only if needed to verify local state.

7. AGENTS.md 와 Skills 체계

AGENTS.md 는 repo 전체 헌법이다. Skill 파일은 특정 작업을 잘하게 만드는 세부 매뉴얼이다. Anthropic 의 개발자 자료도 Skills, Claude Code, MCP, RAG, evaluations 를 별도 역량으로 다루며, Skills 를 특정 작업 수행을 향상시키는 지시 체계로 설명한다 [S5].

7.1 AGENTS.md 의 역할

- AI 가 매번 따라야 할 repo-level operating rules 를 제공한다.
- 테스트 명령, 금지 작업, 파일 구조, 보고 양식을 고정한다.
- 신규 개발자에게도 “이 repo 에서 일하는 방식”을 설명한다.
- 모든 Codex prompt 의 첫 줄에서 참조시킨다.

7.2 AGENTS.md Template

```
# AGENTS.md - Asperitas Agent Development Contract

## Mission
Build reliable, source-grounded, testable AI agents for Asperitas operations.

## Operating Rules
- Make the smallest safe change.
- Preserve backward compatibility.
- Do not delete files unless explicitly approved.
- Never commit secrets, private keys, tokens, or credentials.
- Use internal source hierarchy when answering or generating knowledge outputs.
- Separate facts, inference, and assumptions.

## Repository Structure
- 00_ADMIN: inventories, policies, decision logs
- 01_RAW_SOURCES: raw source material
- 02_REDACTED: sanitized sources
- 03_PROCESSED_KB: processed markdown/chunks/summaries/citations
- src/: production code
- tests/: test suite
- scripts/: verification/eval utilities
- 09_LOGS: decision logs and run reports

## Required Verification
- Run pytest after code changes.
- Run artifact verification after artifact/data changes.
- Run retrieval eval after retrieval/chunking/scoring/generation changes.

## Report Format
1. objective
2. files changed
3. tests run
4. eval metrics before/after
5. risks
6. suggested commit message
```

7.3 Skill 구조

Skill 은 반복 작업을 에이전트가 더 잘 수행하도록 만드는 작은 운영 매뉴얼이다. 예를 들어 retrieval eval, PDF ingestion, regulatory citation, open-source intake, GitHub PR review 는 각각 별도 skill 로 분리할 수 있다.

Skill 이름	목적	언제 사용
retrieval-eval	RAG 검색 변경 후 metric 검증	chunking, scoring, reranking 변경 시
source-ingestion	문서 수집/정리/메타데이터화	새 자료 ingest 전후
regulatory-citation	규제/정부 문서 근거 정리	CITES, Nagoya, LMO, K-BDS 관련 답변
oss-intake	오픈소스 후보 검토	외부 repo 를 참고/도입하기 전
github-release	commit/PR/release note 표준화	MVP 완료 후
failure-taxonomy	오류 유형 분류와 회귀 테스트 설계	eval 실패 분석 시

7.4 Skill Template

```
---
name: oss-intake
description: Evaluate open-source repositories before adapting code or patterns into Asperitas agents.
---

# OSS Intake Skill

## Use when
- A task proposes importing, copying, adapting, or learning from an external GitHub repository.
```

```
## Required checks
1. Identify repository, commit/release, license, and maintainers.
2. Check commercial-use compatibility.
3. Check dependencies, security alerts, and known vulnerabilities.
4. Run candidate in a sandbox if adoption is considered.
5. Prefer pattern adaptation over direct copying.
6. Update OSS Intake Ledger.

## Forbidden
- Do not copy code with missing or incompatible license.
- Do not add dependencies without justification.
- Do not paste secrets or private data into external tools.

## Report
1. candidate summary
2. license/security status
3. architecture fit
4. adopt/reject recommendation
5. required attribution
```

7.5 새 채팅에서 성능향상/스킬추가/워크플로우 구축을 하는 법

본격 개발 채팅과 성능향상 채팅은 분리하는 것이 좋다. 구현 채팅은 변경 범위를 작게 유지해야 하고, 성능향상 채팅은 meta-level 로 workflow, AGENTS.md, skills, prompt library, CI, benchmark 를 개선한다.

새 채팅 유형	목적	입력해야 할 것	산출물
Implementation Chat	특정 MVP 구현	repo 상태, MVP prompt, target files	코드 변경, 테스트, 커밋
Review Chat	결과 검증	screenshots/logs/diff/eval metrics	commit 가능 여부, fix prompt
Skill Chat	반복 작업 자동화	반복되는 작업 설명, 실패 사례	SKILL.md, checklist, prompt template
Workflow Chat	개발 프로세스 개선	현재 bottleneck, 팀 규모, 도구	pipeline, CI, GitHub rules, onboarding docs
OSS Scout Chat	오픈소스 탐색	필요 기능, 언어, license 제약	candidate list, intake ledger, adopt/reject

8. 검증 체계: AI 코딩의 생명선

AI가 작성한 코드는 “보기에 그럴듯한 것”과 “시스템에 안전한 것”이 다르다. 따라서 모든 개발 단계는 검증 명령으로 끝나야 한다. 특히 RAG/agent 시스템은 단순 unit test 만으로 충분하지 않다. 검색 품질, citation fidelity, source priority, latency, 비용, security, data leakage 를 함께 봐야 한다.

8.1 테스트 계층

계층	목적	예시
Unit Test	작은 함수가 의도대로 동작하는지 검증	chunk parser, metadata schema, scorer
Integration Test	모듈 간 연결 검증	ingest -> chunk -> retrieval
Artifact Verification	생성된 파일/데이터 구조 검증	chunks.jsonl schema, registry count
Retrieval Eval	검색 품질 수치화	source match @k, evidence, section match
Generation Eval	답변 정확도와 citation 검증	grounded answer, refusal when missing evidence
Regression Test	기존 PASS/FAIL behavior 보존	이전 bug fixture 재실행
Security Test	secret, unsafe dependency, injection 위험 확인	secret scan, dependency review
Human Review	비즈니스/규제/도메인 판단	COO/SME approval

8.2 MVP 별 통과 기준 예시

MVP 유형	필수 검증	추가 검증
Chunking 변경	pytest, chunk section audit, artifact verify	retrieval eval before/after
Retrieval scoring 변경	pytest, retrieval eval	metric degradation explanation, tie-break audit
LLM generation 추가	pytest, prompt fixture eval	citation fidelity, hallucination refusal cases
UI/CLI 추가	pytest, smoke test	screenshot or command transcript
Data ingestion 변경	artifact verify, schema validation	source hierarchy and metadata audit
Dependency 추가	pytest, dependency review	license/security ledger
Workflow/Skill 변경	docs lint/review	dry run with sample task

8.3 실패 시 행동 규칙

- 실패 로그를 보존한다. 성공 로그보다 실패 로그가 더 중요하다.
- 실패 원인을 test, code, data, environment, spec ambiguity 로 분류한다.
- Codex 에게 “실패만 고쳐라”라고 명시한다. 범위 확장을 금지한다.
- 테스트를 삭제하거나 약화시키지 않는다. 테스트가 정말 틀린 경우에는 근거를 남기고 수정한다.
- 수정 후 failing test 만 먼저 돌리고, 그다음 full pytest 를 돌린다.
- eval metric 이 악화되었으면 이유와 trade-off 를 decision log 에 남긴다.

8.4 Eval Metrics 예시

Metric	의미	나쁜 신호
Source Match @3/@5	정답 source 가 상위 검색 결과에 포함되는지	새 scoring 후 하락
Priority Match	source hierarchy 가 반영되는지	내부 소스보다 낮은 우선순위 자료가 먼저 나올
Evidence Match	답변 근거 chunk 가 검색되는지	정답은 맞는데 근거가 약함
Section Match	문서 내 정확한 section 을 찾는지	같은 문서의 엉뚱한 section 검색
Citation Fidelity	출처가 실제 주장과 맞는지	citation 이 문장을 뒷받침하지 못함
Refusal Accuracy	근거 부족 시 답변 거절하는지	없는 정보를 만들어냄
Latency/Cost	응답 시간과 비용	품질 향상 대비 비용 폭증

9. 신규 개발자 온보딩 프로세스

신규 개발자는 곧바로 main 에 코드를 밀어 넣지 않는다. 먼저 문서와 루프를 이해하고, 작은 read-only task 와 test-only task 를 통해 repo 의 검증 문화를 익힌다.

9.1 Day 0 - 접근권한과 환경

- GitHub 계정 초대 및 권한 확인. 초기에는 read 또는 limited write 권한.
- repo clone 또는 GitHub Desktop 연결.
- Python/Node/IDE/Codex/VS Code/Cursor 등 프로젝트별 도구 설치.
- API 키는 개인 로컬 환경변수 또는 회사 secret manager 로만 관리. repo 에 저장 금지.
- AGENTS.md, README, decision logs, latest MVP report 읽기.

Developer local setup checklist

```
git clone [private repo URL]
cd asperitas-agent
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
# macOS/Linux
source .venv/bin/activate
python -m pip install -U pip
pip install -e .[dev]
pytest
python scripts/verify_artifacts.py
```

9.2 Day 1 - Read-only repo map 작성

첫날 과제는 코딩이 아니라 repo 지도를 만드는 것이다. 신규 개발자는 Codex 또는 ChatGPT 도움을 받아 구조를 읽고, 어떤 파일이 어떤 역할인지 요약한다.

Use AGENTS.md and relevant skills.

Task:

Create a read-only repository map for onboarding. Do not edit files.

Report:

1. top-level folders and their purpose
2. production modules and their responsibility
3. test modules and what they cover
4. scripts and verification commands
5. current MVP status inferred from docs/logs
6. questions or risks for a new developer

9.3 Day 2-3 - Test-only contribution

- 작은 regression test 또는 fixture 를 추가한다.
- 기존 기능을 깨지 않고 테스트 문화에 익숙해지는 단계다.
- PR 에서 “왜 이 테스트가 필요한지”를 설명하게 한다.

9.4 Day 4-5 - Documentation/Skill contribution

- 반복 작업 하나를 골라 .agents/skills 로 분리한다.
- 예: artifact verification skill, retrieval eval skill, GitHub commit skill.
- 실제 Codex task 에 적용해 dry run 한다.

9.5 Day 6-7 - 첫 MVP 구현

- 작은 feature branch 에서 첫 MVP 를 구현한다.
- PR 템플릿을 채우고 reviewer 가 승인한다.
- 테스트 통과, artifact verify, eval metric 보고까지 완료해야 merge 가능하다.

9.6 신규 개발자 금지사항

- main 에 대형 리팩터링을 직접 푸시하지 않는다.
- 테스트 실패를 무시하고 커밋하지 않는다.
- 라이선스 확인 없이 외부 코드를 복사하지 않는다.
- API 키나 회사 자료를 공개 도구에 업로드하지 않는다.
- AI 가 만든 설명을 사실로 단정하지 않는다. repo 와 로그로 확인한다.
- 문서화 없이 dependency 를 추가하지 않는다.

10. 에이전트 유형별 개발 레시피

Asperitas 는 하나의 RAG Agent 에서 끝나면 안 된다. 목표는 독자적인 Agent Factory 다. 각 에이전트는 도메인은 다르지만 같은 개발 공정을 따라야 한다.

Agent	목적	핵심 Eval
RAG Knowledge Agent	내부 자료/논문/규제 문서 검색과 근거 기반 답변	retrieval precision, citation fidelity, source hierarchy
Literature Intelligence Agent	논문 탐색, 요약, 연구 공백 분석	paper recall, claim extraction, citation grounding
Regulatory Compliance Agent	CITES/Nagoya/LMO/K-BDS/IRIS/NRI 규제 추적	latest source, jurisdiction mapping, refusal on uncertainty
DBTL Planning Agent	Design-Build-Test-Learn 실험 계획 보조	experiment traceability, hypothesis clarity, biosafety gate
Biofoundry Ops Agent	실험 자동화 SOP, 장비/시약/샘플 흐름 관리	SOP consistency, audit trail, deviation detection
IP/Patent Scout Agent	특허/논문/경쟁사 moat 분석	novelty search quality, claim mapping, prior art coverage
Investor/Market Intelligence Agent	VC/파트너/시장 자료 수집과 IR 보조	source credibility, update recency, thesis alignment
Talent/Onboarding Agent	신규 인력 교육, 업무 배정, 문서 안내	completion rate, onboarding speed, access safety
Security/Secrets Agent	repo secret, dependency, prompt injection 리스크 감시	zero secrets committed, vulnerability response time
Decision Log Agent	회의/개발/전략 의사결정 기록 구조화	decision traceability, action owner clarity

10.1 모든 에이전트의 공통 Skeleton

```
agent_name/  
  README.md  
  AGENTS.md  
  .agents/  
    skills/  
      [task-skill]/SKILL.md  
  src/  
    agent_core.py  
    tools.py  
    schemas.py  
    retrieval.py  
    evaluation.py  
  tests/  
    test_agent_core.py  
    test_schemas.py  
    test_evaluation.py  
  scripts/  
    verify_artifacts.py  
    run_eval.py  
  data/  
    fixtures/  
    eval_sets/  
  docs/  
    architecture.md  
    decision_log.md  
    onboarding.md  
  .github/  
    workflows/ci.yml  
    pull_request_template.md
```

10.2 RAG Agent 다음 단계 예시

Roadmap 단계	목표	검증
MVP-012 Failure Taxonomy	검색 실패를 유형화해 개선 방향을 명확히 함	failure labels distribution, regression cases
MVP-013 Embedding Baseline	TF-IDF 외 embedding retrieval 비교	source match/evidence/latency before-after
MVP-014 Reranker	상위 candidate 재정렬	precision @3, section match

MVP-015 Citation Generator	답변 생성과 citation 연결	citation fidelity eval
MVP-016 Refusal Policy	근거 부족 시 안전한 거절	false answer rate 감소
MVP-017 Admin UI/CLI	사용자가 쉽게 질의/검증	smoke test, usability checklist
MVP-018 CI Eval Gate	PR 에서 eval regression 자동 차단	GitHub Actions pass/fail gate

10.3 에이전트별 Agent Brief 예시

Agent Brief: Regulatory Compliance Agent

Business Objective:
Reduce regulatory uncertainty in biodiversity-derived biotech operations.

Primary User:
COO, BD, research operations, compliance reviewer.

Input Sources:
Internal policies, government portals, CITES/Nagoya/LMO/K-BDS/IRIS/NRI documents, official updates.

Expected Output:
Jurisdiction-specific compliance checklist with citations, uncertainty flags, and required next actions.

MVP Scope:
Read-only retrieval and citation-backed summary for selected Korean and international regulatory documents.

Non-goals:
No legal advice, no final compliance approval, no automated submission.

Success Metrics:
Citation fidelity, source priority match, refusal accuracy on unsupported claims, update recency.

11. 성능향상/워크플로우/파이프라인 전용 루프

본격적인 기능 구현과 별개로, 주기적으로 개발 시스템 자체를 개선해야 한다. 이것이 “AI 팀”처럼 작동하는 핵심이다. 코드를 만드는 agent 와, agent 를 더 잘 일하게 만드는 meta-agent 작업을 분리한다.

11.1 Meta-development 루프

Meta-development Loop

1. Observe bottleneck
 - 반복되는 prompt 작성
 - 잦은 테스트 실패
 - 평가 지표 해석 어려움
 - GitHub commit/PR 실수
 - 오픈소스 검토 누락
2. Convert bottleneck into Skill or Workflow
 - AGENTS.md rule
 - .agents/skills instruction
 - prompt template
 - GitHub Action
 - checklist
3. Dry run on one MVP
4. Measure friction reduction
5. Commit workflow asset
6. Train next developer with updated playbook

11.2 어떤 것은 코드가 아니라 Skill 로 해결한다

문제	나쁜 해결	좋은 해결
매번 보고 형식이 다름	매번 말로 다시 설명	AGENTS.md report format 고정
오픈소스 검토를 까먹음	개발자 기억에 맡김	oss-intake Skill + Ledger 템플릿
retrieval eval 명령을 잊음	실패 후 수동 확인	retrieval-eval Skill + CI gate
신규 개발자가 repo 구조 모름	구두 설명 반복	onboarding.md + repo-map prompt
테스트 실패 원인 분석 느림	전체 리팩터링 지시	failure-taxonomy Skill
커밋 메시지 들쭉날쭉	아무 메시지 작성	commit convention + PR template

11.3 월간 Agent Factory Review

- 이번 달 만들어진 MVP 수, merged PR 수, rollback 수를 본다.
- 가장 많이 실패한 테스트/평가 항목을 본다.
- 반복되는 지시문을 Skill 로 승격한다.
- 오픈소스 후보와 도입 결정을 정리한다.
- 다음 달에는 코드 기능이 아니라 pipeline 개선 MVP 를 1 개 이상 잡는다.
- 신규 개발자 onboarding friction 을 측정한다. 문서만 보고 setup 가능한지 확인한다.

11.4 Team Operating Cadence

주기	회의/작업	산출물
Daily 또는 작업 시작 전	Repo status + current MVP 확인	오늘의 Codex prompt, risk note
MVP 완료 시	Result review	commit, decision log, next prompt
주 1 회	Agent pipeline review	bottleneck list, skill candidates
월 1 회	Agent Factory board review	roadmap, metrics, developer productivity
분기 1 회	Architecture and security review	dependency cleanup, license audit, CI improvement

12. CEO-grade Engineering Doctrine

이 장은 실제 특정 CEO의 발언을 인용하는 것이 아니라, OpenAI/Anthropic 급 AI 인프라 회사를 지향한다면 내부 개발자에게 내려야 할 수준의 운영 지시를 Asperitas 맥락으로 재구성한 것이다.

12.1 지시 1 - 작은 변경이 빠른 변경이다

대형 리팩터링은 멋있어 보이지만 검증 비용을 폭발시킨다. AI 코딩에서는 변경량이 커질수록 hallucinated architecture, hidden regression, 테스트 누락이 증가한다. 따라서 Codex 에게는 항상 smallest safe change 를 요구한다.

12.2 지시 2 - Eval 이 없는 Agent 는 제품이 아니다

AI Agent 는 데모가 아니라 반복 측정 가능한 시스템이어야 한다. 검색 agent 는 retrieval metric 이 있어야 하고, literature agent 는 citation accuracy 가 있어야 하며, compliance agent 는 source recency 와 refusal accuracy 가 있어야 한다. Eval 이 없으면 “좋아진 것 같다”는 착각만 남는다.

12.3 지시 3 - GitHub 는 조직 기억장치다

commit, PR, issue, decision log 는 나중에 들어오는 개발자에게 회사의 사고 과정을 전송하는 장치다. 코드는 결과이고, decision log 는 왜 그렇게 만들었는지에 대한 근거다. 둘 중 하나만 있으면 조직 지식은 불완전하다.

12.4 지시 4 - 오픈소스는 속도지만, 무검증 오픈소스는 부채다

좋은 오픈소스를 빠르게 탐색하되, license/security/dependency/maintenance 검증 없이 회사 core repo 에 넣으면 안 된다. 특히 생물 정보, RAG, agent orchestration 분야는 빠르게 변하기 때문에 “인기 repo”보다 “검증 가능한 repo”가 중요하다.

12.5 지시 5 - 인간은 승인권, AI 는 반복 실행권

AI 에게 많은 실행을 맡길수록 인간의 역할은 줄어드는 것이 아니라 더 높은 수준으로 올라간다. 인간은 목표, trade-off, regulatory risk, customer value, IP boundary 를 판단해야 한다. AI 는 빠른 구현과 검증 반복을 맡는다.

12.6 지시 6 - 개발 파이프라인 자체를 제품화하라

Asperitas 가 진짜 강해지려면 하나의 에이전트를 만드는 데서 끝나지 않고, 에이전트를 계속 찍어내는 Agent Factory 를 만들어야 한다. AGENTS.md, Skills, prompt library, CI, eval sets, decision logs, onboarding docs 가 바로 그 공장 설비다.

12.7 전략 필터

필터	질문	통과 기준
Scalability	이 방식이 다음 에이전트 10 개에도 반복 가능한가?	template, tests, docs, CI 로 재사용 가능
Moat	우리만의 데이터/평가/운영 지식이 축적되는가?	internal source hierarchy 와 eval set 에 반영
Biosafety/Compliance	규제/보안/라이선스 리스크를 낮추는가?	source/citation/license/security gate 존재
Velocity	다음 MVP 까지 마찰을 줄이는가?	prompt reuse, skill reuse, small PR
Reliability	실패를 빨리 발견하고 복구 가능한가?	tests, rollback, decision log

13. 복붙용 명령어·체크리스트

13.1 커밋 전 체크리스트

Pre-commit Checklist

```
[ ] git status checked
[ ] changed files reviewed
[ ] no unexpected deletions
[ ] no secrets/API keys
[ ] no unreviewed dependency
[ ] pytest passed
[ ] artifact verification passed if applicable
[ ] retrieval/eval metrics checked if applicable
[ ] docs updated if behavior changed
[ ] decision log updated if MVP-level change
[ ] commit message includes MVP or change type
```

13.2 GitHub Desktop 기준

17. GitHub Desktop 열기
18. Current Repository 가 올바른 repo 인지 확인
19. Current Branch 확인. 혼자 MVP 초기 개발이면 main 가능, 팀 개발이면 feature branch 사용
20. Fetch origin 클릭
21. 변경 파일 목록 확인
22. 불필요한 파일이 있으면 커밋 제외 또는 되돌리기
23. Summary 작성: MVP-xxx: short action
24. Description 에 tests/eval 결과 요약
25. Commit to [branch] 클릭
26. Push origin 클릭
27. No local changes / up to date 상태 확인

13.3 CLI 기준

```
git fetch origin
git status --short
git diff --stat
pytest
python scripts/verify_artifacts.py
# if retrieval/chunking/scoring/generation changed
python scripts/run_retrieval_eval.py

git add [files]
git commit -m "MVP-xxx: implement [short description]"
git push origin [branch]
git status
git log -1 --oneline
```

13.4 Codex 에게 “검증만” 시킬 때

Use AGENTS.md and relevant skills.

Task:
Verify the current repository state only. Do not edit files.

Run or inspect:

- git status
- pytest
- artifact verification
- retrieval eval if applicable
- latest commit and branch sync

Report:

1. branch and HEAD
2. working tree status
3. tests/eval results
4. whether it is safe to commit/push
5. any blocking risk

13.5 Codex 에게 “다음 MVP 후보”를 뽑게 할 때

Use AGENTS.md and relevant skills.

Task:

Recommend the next 3 MVP candidates based on the current repo, tests, eval logs, and decision logs. Do not edit files.

Rank each by:

- user/business value
- implementation difficulty
- regression risk
- eval impact
- dependency/security risk

Report:

1. recommended next MVP
2. why it should be first
3. rejected alternatives
4. exact Codex implementation prompt for the chosen MVP

14. Risk Register

리스크	발생 원인	예방 장치	대응
AI hallucinated code	repo 맥락 부족, prompt 불명확	AGENTS.md, target files, tests	diff review, failing test fix prompt
대형 변경으로 regression	범위 잠금 실패	smallest safe change, PR size limit	revert, split PR
오픈소스 라이선스 위반	license 확인 없이 복사	OSS Intake Ledger	코드 제거, 재구현, 법무 검토
Secret leakage	API 키 커밋, config 파일 노출	secret scan, .gitignore, env vars	rotate keys, purge history
Eval 지표 악화	scoring/chunking 변경 영향	before/after eval	rollback or tune
문서와 코드 불일치	behavior 변경 후 docs 미수정	docs update gate	docs patch PR
신규 개발자 혼선	온보딩 문서 부족	repo map, playbook, skills	pair review, onboarding issue
규제/바이오 안전 리스크	근거 없는 compliance 답변	source hierarchy, refusal policy	SME review, disclaimer, escalation
Dependency supply-chain attack	검증 안 된 package 도입	dependency review, pinning, minimal deps	remove package, audit, update lockfile
Hidden data provenance issue	raw source 출처 불명	metadata.csv, citation ledger	quarantine source, re-ingest

15. Glossary

용어	뜻
Vibe Coding	자연어와 AI를 이용해 빠르게 코드를 만들되, 이 플레이북에서는 반드시 테스트와 GitHub 검증을 붙인 방식으로 정의함
Codex	OpenAI의 coding agent. CLI/IDE/Web/App 흐름에서 코드 읽기, 편집, 실행, 버그 수정, PR 보조 등에 사용
AGENTS.md	repo-level AI 작업 헌법. 모든 AI 작업의 기본 규칙
Skill	특정 반복 작업을 더 잘 수행하게 하는 세부 지시문
MVP	작고 검증 가능한 제품/기능 단위
Preflight	구현 전 읽기 전용 영향 분석
Artifact Verification	생성된 데이터/파일이 schema와 registry 기준을 만족하는지 확인
Retrieval Eval	RAG 검색 품질을 정량 평가하는 절차
Decision Log	왜 이 변경을 했는지 남기는 운영 기록
Protected Branch	중요 브랜치에 삭제/force push/미검증 merge를 제한하는 GitHub 설정
Dependency Review	새 dependency가 보안/라이선스/나이/사용 현황상 위험한지 PR에서 확인하는 절차
Source Hierarchy	내부 자료, 공식 문서, 정부/규제 문서 등 출처 신뢰도 순위

16. Source References

아래 자료는 이 플레이북의 도구·운영 전제를 확인하기 위해 참고한 공식 또는 1 차에 가까운 문서다. 제품 기능은 시간이 지나며 바뀔 수 있으므로 신규 개발자는 실제 도입 전 최신 문서를 다시 확인해야 한다.

- [S1] OpenAI Codex CLI GitHub repository. Codex CLI is described as a coding agent from OpenAI that runs locally and includes CLI/IDE/app/web paths.
- [S2] OpenAI Codex web/cloud documentation. Codex is described as a coding agent that can read, edit, and run code and can work in a cloud environment.
- [S3] OpenAI Prompt Engineering Guide. Prompt engineering is framed as writing effective instructions to obtain desired outputs consistently.
- [S4] OpenAI Help Center: Connecting GitHub to ChatGPT. GitHub connection can let ChatGPT pull live repository data and reason over code/docs.
- [S5] Anthropic Build with Claude / Developer education page. Covers Claude Code, Skills, MCP, tool use, RAG, evaluations, prompt engineering, and related development resources.
- [S6] GitHub Docs: Protected branches. Branch protection can restrict deletes/force pushes and require status checks or review workflows.
- [S7] GitHub Docs: Licensing a repository. A repository needs a license for others to freely use, change, and distribute software.
- [S8] GitHub Docs: Dependency review. Helps understand dependency changes and security impact in pull requests.
- [S9] GitHub Docs: Code scanning. Used to analyze repositories for vulnerabilities and code errors.
- [S10] GitHub Docs: Supply chain security. Dependency review, Dependabot, and malware alerts support dependency risk control.

17. Final Operating Rule

최종 규칙

Asperitas의 모든 에이전트 개발은 “AI가 코드를 썼다”가 아니라 “AI가 작성한 변경이 테스트, 평가, 보안, GitHub 이력, 의사결정 기록을 통과했다”일 때만 완료로 인정한다.